

C2.1 – Architecture logicielle

1. Pattern architectural retenu

****Monolithe modulaire full-stack Next.js 14** :**

- ****Couche présentation**** : pages et composants React (`app/`, `frontend/components/`), marqués `use client` lorsque l'interactivité l'exige.
- ****Couche API**** : Route Handlers (`app/api/\*\*/route.js`) – point d'entrée HTTP, validation session, orchestration.
- ****Couche métier**** : services (`backend/services/`, `app/lib/`) – logique réutilisable, appels externes, règles de campagne et scoring.
- ****Couche données**** : Supabase client (anon + service role selon le contexte), schéma SQL + RLS.

Ce pattern évite la dispersion de la logique métier dans les composants UI tout en gardant un seul artefact déployable.

2. Diagramme C4 – Niveau 1 (Contexte)

```
```mermaid
flowchart LR
 candidat[Candidat]
 recruteur[Recruteur]
 careerai[CareerAI_Next_js]
 supabase[(Supabase)]
 groq[Groq_IA]
 resend[Resend]
 offres[Agregateurs_offres]

 candidat -->|HTTPS| careerai
 recruteur -->|HTTPS| careerai
 careerai --> supabase
 careerai --> groq
 careerai --> resend
 careerai --> offres
```
```

3. Diagramme C4 – Niveau 2 (Conteneurs)

```
```mermaid
flowchart TB
 subgraph browser [Navigateur_ou_APK]
 ui[React_UI]
 end
 subgraph nextjs [Application_Next_js]
 pages[App_Router_pages]
 api[API_Routes]
 mw[middleware_js]
 svc[Services_metier]
 end
 subgraph supa [Supabase]
 auth[Auth]
 db[(PostgreSQL_RLS)]
 storage[Storage_documents]
 end
 ext[APIs_externes_Groq_Resend_Adzuna]
```

```
ui --> pages
pages --> api
mw --> api
api --> svc
api --> auth
svc --> db
svc --> storage
svc --> ext
...

```

#### ## 4. Séquence – Authentification

```
```mermaid
sequenceDiagram
    participant U as Utilisateur
    participant UI as Page_signup
    participant SB as Supabase_Auth
    participant MW as middleware_js

    U->>UI: email + mot de passe
    UI->>SB: signUp
    SB-->>UI: session ou email confirmation
    U->>UI: navigation app
    UI->>MW: requête cookie session
    MW->>SB: getSession
    alt session valide
        MW-->>UI: accès routes protégées
    else pas de session
        MW-->>UI: redirect /welcome
    end
...
---
```

Fichiers : [app/auth/signup/page.js](../app/auth/signup/page.js),
[middleware.js](../middleware.js).

5. Séquence – Exécution campagne (run-now)

```
```mermaid
sequenceDiagram
 participant U as Candidat
 participant API as POST_api_campaigns_run_now
 participant CS as CampaignService
 participant DB as Supabase
 participant R as Resend_ou_Puppeteer

 U->>API: POST (cookie session)
 API->>DB: vérifier session + campagnes actives
 API->>CS: runCampaignDay(campaignId)
 CS->>DB: lire candidate_profiles
 CS->>CS: fetchAllJobsForProfile + matchOffersToProfile
 CS->>R: envoi candidatures
 CS->>DB: insert campaign_applications
 API-->>U: JSON résultats + offres à consulter
...

```

Fichiers : [app/api/campaigns/run-now/route.js](../app/api/campaigns/run-now/route.js), [backend/services/CampaignService.js](../backend/services/CampaignService.js).

---

## ## 6. Interfaces entre composants

### ### 6.1 UI !' API ( REST interne )

Le client utilise [frontend/lib/api.js](../frontend/lib/api.js) (`fetch` + cookies). Exemples :

Méthode	Endpoint	Rôle
GET	`/api/campaigns`	Liste des campagnes
POST	`/api/campaigns`	Créer une campagne
POST	`/api/campaigns/run-now`	Exécution immédiate
POST	`/api/recruiter/candidates`	Ajouter candidat (multipart)
POST	`/api/recruiter/quizzes/{id}/send`	Envoyer quiz

Spécification OpenAPI : [05-api-openapi.yaml](05-api-openapi.yaml).

### ### 6.2 API !' Services

Les routes importent les services métier plutôt que d'implémenter la logique inline :

- `CampaignService.runCampaignDay` – orchestration campagne.
- `RankingService` – scores et classements recruteur.
- `chat.js` / `AIService` – appels Groq.

### ### 6.3 Services !' Supabase

- **\*\*Client utilisateur\*\*** (`createRouteHandlerClient`) : opérations soumises au RLS.
- **\*\*Service role\*\*** (`SUPABASE\_SERVICE\_ROLE\_KEY`) : cron, opérations admin limitées (upload storage recruteur, ranking).

---

## ## 7. Sécurité logicielle

Mécanisme	Implémentation
Auth session	Cookies Supabase + middleware
RLS	Politiques par `user_id` / `recruiter_id`
Secrets serveur	Variables `.env.local` uniquement côté Node
Cron	Header `Authorization: Bearer CRON_SECRET`
Quiz public	Token opaque dans URL, pas d'auth globale

---

## ## 8. Scalabilité et performance

Aspect	Approche
API stateless	Pas de session serveur Next ; scaling horizontal possible (Vercel/Render)
Plafonds métier	`max_applications_per_day`, durée campagne
Traitements longs	`maxDuration` sur routes campagne ; exécution asynchrone via cron
Traçabilité	Tables `campaign_runs`, `campaign_applications`
Cache	Pas de cache distribué V1 ; évolution possible (Redis) si volume

---

## ## 9. Structure des répertoires

```

...
app/ # App Router (pages + API)
 api/ # 33 route handlers
 auth/ # login, signup, etc.
frontend/
 components/ # UI React
 contexts/ # Auth, Toast, i18n
 hooks/ # useChat, etc.
backend/
 services/ # Logique métier lourde
 lib/ # Utilitaires Supabase
supabase-migrations/ # Évolutions schéma
docs/ # Documentation grille RNCP
...
```

---

## ## 10. Évolutions envisagées

- File d'attente (Bull/Redis) pour les envois de campagne en pic de charge.
- OpenAPI étendue à l'ensemble des 33 routes.
- Tests E2E (Playwright) sur parcours inscription et campagne.